Python for Network Engineers

From Python Fundamentals to Real-World Network Automation

Study Guide · Hands-on Lab Manual · Reference · Practice Workbook

Version 0.1 (in development)

Author: (your name)

Organization: (your organization)

2026

About This Guide

This guide teaches a network engineer how to use Python to automate real network work, starting from the absolute basics and building up to production-quality automation tools. It is designed to serve four purposes at once: a study guide you read, a hands-on lab manual you work through, a reference you come back to, and a practice workbook with exercises and solutions.

Who this is for

Network engineers with or without any programming background. If you know what a VLAN, an interface, and a BGP neighbor are, and you can use a device command line, the networking side is covered. This course supplies the Python.

How to use it

Do Chapter 0 first to set up your environment. Then work through the chapters in order. Read the concept, study the code walkthrough, then do the lab with your own hands. Try each exercise and challenge before opening its solution. The solutions are stored separately for exactly that reason.

Lab environment

Labs are tiered so you are never blocked by a lack of hardware. Many early labs run on mock data with no devices at all. Later labs use free sandboxes, containers, or emulators, and each lab tells you what it needs.

Contents

Chapter 0: Prerequisites and Environment Setup

> **Part 1: Python Foundations for Network Engineers** This is the one chapter you do before any coding. By the end your computer has Python, an editor, and Git installed, you have a GitHub account, and your course project folder is created and saved to GitHub. Everything after this runs inside the environment you build here.

Learning objectives

After this chapter you will be able to:

- Install the latest Python (3.14) and confirm it works on Windows, macOS, or Linux.
- Explain what pip and a virtual environment are, and why every project should have its own.
- Create and activate a virtual environment, and install packages into it.
- Set up VS Code and IPython, and run code interactively line by line.
- Create a GitHub account and install and configure Git.
- Turn your project folder into a Git repository, make your first commit, and push it to GitHub.

Why this matters to a network engineer

You already treat device configuration with care. You keep backups, you note what changed, and you can roll back when a change goes wrong. Your automation code deserves the same discipline, because a script that pushes config to fifty devices is more dangerous than any single device change.

A clean environment gives you three things. First, your tools are isolated, so a package you install for one project cannot quietly break another. Second, your work is version controlled, so every change to a script or a template is saved with a message and a timestamp and can be undone. Third, your work lives on GitHub, so it survives a laptop failure and can be shared with your team. Setting this up once, properly, saves you from a long list of confusing problems later.

Concept

A working Python environment is a few separate pieces that work together.

The interpreter is the `python` program itself. It reads your code and runs it. We install the latest version, 3.14, so you get the current syntax and the newest libraries.

pip is Python's package installer. Most useful network libraries, like netmiko and napalm, are not built into Python. pip downloads and installs them for you from the Python Package Index.

A virtual environment is a private copy of Python and its packages for one project. Without it, everything you install goes into one shared pile, and two projects that need different versions of the same library will fight. With it, each project gets its own clean box. You will create one virtual environment for this course.

An editor is where you write code. We use VS Code, which is free, works the same on all three operating systems, and has good Python support. IPython is an improved interactive Python shell that lets you type one line, see the result, and type the next. That is the read-along style this book uses.

Git is a version control tool that runs on your machine and records the history of your files. GitHub is a website that stores a copy of your Git history in the cloud. Git is the tool, GitHub is the place. You need both.

Network example

Think about a Jinja2 template that generates interface configuration. Over a few weeks you tweak it many times. Without version control, you end up with files named `template_final`, `template_final_v2`, and `template_really_final`, and no idea which one is correct. With Git, there is one file, and its full history is saved. You can see exactly what changed between last Tuesday and today, and if a change broke your generated config, you can roll back in seconds. That is the everyday value of the setup in this chapter.

How it works

When you run `python script.py`, your operating system finds the Python interpreter, which reads the file and executes it. When you activate a virtual environment first, your shell is told to use that environment's private Python and packages instead of the system-wide ones. When you run a Git command, Git updates a hidden `.git` folder inside your project that holds the complete history. When you run `git push`, Git sends the new history to GitHub over the network.

Visual explanation

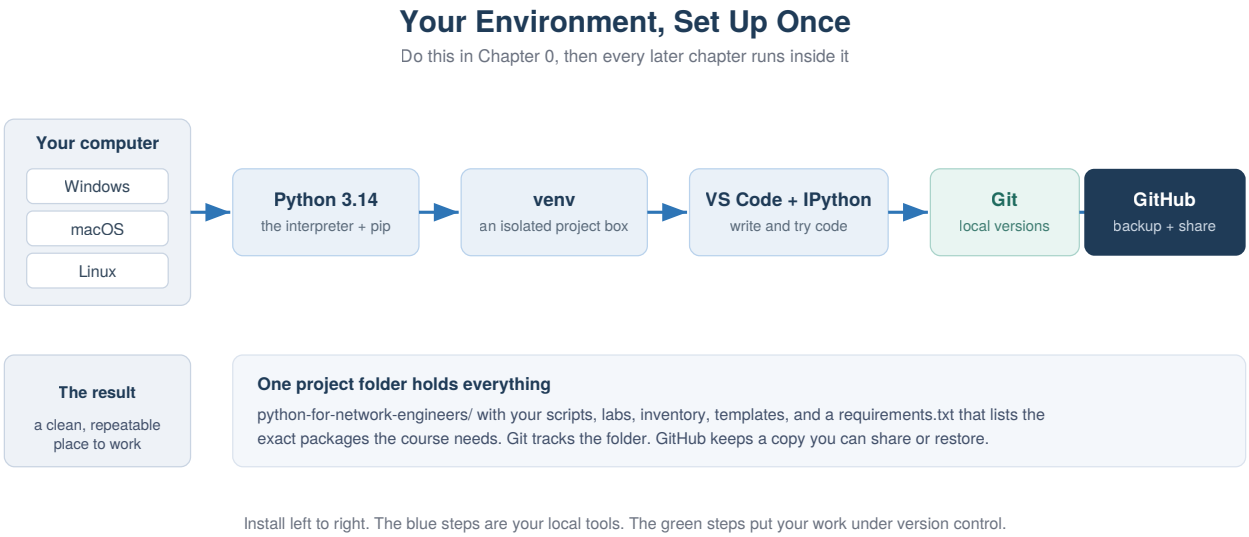

Figure 0.1: The pieces you install in this chapter and how they connect. The blue steps are your local tools. The green steps put your work under version control.

Figure 0.2: Git has three local stages, working directory, staging area, and local repository, moved along with add and commit. Push sends the saved history up to GitHub, and pull or clone brings it back down.

Setup walkthrough

Follow the section for your operating system. Commands you type are shown in code blocks. Lines that start with `#` are comments for you to read, not to type.

Part A: Install Python 3.14

Windows. Go to python.org, open Downloads, and get the latest Python 3.14 Windows installer (64 bit). Run it. On the very first screen, tick the box that says "Add python.exe to PATH" before you click Install. This one checkbox prevents the most common Windows problem, where the `python` command is not found later. Finish the install, then open a new PowerShell window and check the version:

```
python --version
```

You should see `Python 3.14.x`. Windows also installs a launcher called `py`, so `py --version` works too.

macOS. The Python that ships with macOS is old and meant for the system, not for your work. Install a fresh one. The simplest way is the python.org macOS installer: download the latest 3.14 universal installer and run it. Then open Terminal and check:

```
python3 --version
```

If you use Homebrew, `brew install python@3.14` also works. On macOS the command is `python3`, not `python`.

Linux (Debian or Ubuntu). Use the package manager. On recent releases:

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
python3 --version
```

If your distribution does not yet package 3.14, the deadsnakes PPA or pyenv can install it, but any 3.11 or newer will work for the whole course. On Linux the command is `python3`.

> **Important note:** From here on, where this book writes `python`, use `python` on Windows and `python3` on macOS and Linux. The same applies to `pip` versus `pip3`. Once your virtual environment is active, `python` and `pip` work everywhere, which is one more reason to always work inside it.

Part B: Confirm pip and upgrade it

pip comes with Python. Confirm it and upgrade it to the latest version:

```
python -m pip install --upgrade pip
```

Using `python -m pip` rather than a bare `pip` guarantees you are upgrading the pip that belongs to the Python you just installed, which avoids another common mix-up.

Part C: Create and activate a virtual environment

Make the course folder and a virtual environment inside it. Pick a place you can find easily, such as your home folder.

Windows (PowerShell):

```
cd $HOME
mkdir python-for-network-engineers
cd python-for-network-engineers
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If activation is blocked with a message about scripts being disabled, run this once, then activate again:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

macOS and Linux (Terminal):

```
cd ~
mkdir python-for-network-engineers
cd python-for-network-engineers
python3 -m venv .venv
source .venv/bin/activate
```

When it is active, your prompt shows `(.venv)` at the start of the line. That is how you know you are working inside the environment. To leave it later, type `deactivate`. You do not delete it, you just turn it off, and you activate it again next time you work on the course.

> **Best practice:** Activate the virtual environment every time you sit down to work. If you install a package and later cannot import it, the first thing to check is whether `(.venv)` is showing in your prompt.

Part D: Install VS Code and IPython, and try IPython

Download VS Code from code.visualstudio.com and install it for your operating system. Open it, go to the Extensions view, and install the "Python" extension from Microsoft. VS Code has a built-in terminal (View menu, then Terminal) where you can run all the commands in this book.

Now install IPython into your active environment and start it:

```
pip install ipython
ipython
```

You are now in an interactive shell. Type one line at a time and see each result immediately. This is the read-along style used throughout the book:

```
In [1]: hostname = "core-sw-01"

In [2]: vlans = [10, 20, 30]

In [3]: len(vlans)
Out[3]: 3

In [4]: f"{hostname} carries {len(vlans)} VLANs"
Out[4]: 'core-sw-01 carries 3 VLANs'
```

Type `exit` to leave IPython. You will use it constantly to try small pieces of code before putting them into a script.

Part E: Create a GitHub account

Open github.com in your browser and click Sign up. Use an email you will keep, pick a username (a clean, professional handle is worth it, since colleagues and employers see it), and set a strong password. Verify your email when GitHub sends the confirmation.

Then turn on two-factor authentication. In your account settings, under Password and authentication, enable two-factor authentication using an authenticator app. This protects the account that will hold your code, and GitHub increasingly requires it anyway.

> **Warning:** GitHub no longer lets you push code using your account password. When Git asks for a credential later, you use a personal access token or an SSH key, not your password. We set this up in Part G. This trips up almost everyone the first time, so expect it.

Part F: Install and configure Git

Windows. Download Git for Windows from git-scm.com and run the installer. The default options are fine. This also gives you Git Bash, an alternative terminal, but you can keep using PowerShell.

macOS. Installing the Xcode command line tools gives you Git:

```
xcode-select --install
```

Or `brew install git` if you use Homebrew.

Linux (Debian or Ubuntu):

```
sudo apt install git
```

Now tell Git who you are. This name and email are stamped on every commit you make:

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Use the same email you used for GitHub, so your commits are linked to your account.

Part G: Create the repository, first commit, and push

You already have the `python-for-network-engineers` folder from Part C. Turn it into a Git repository and make your first save. Make sure your virtual environment is active and you are inside the folder.

First, create two small files. A `requirements.txt` lists the packages the course uses, and a `.gitignore` tells Git which files to leave out of version control (you never commit the virtual environment or secrets).

Create `requirements.txt` with this line:

```
ipython
```

Create `.gitignore` with these lines:

```
.venv/
__pycache__/
.env
```

Now initialize Git, stage everything, and commit:

```
git init
git add .
git commit -m "Initial commit: course project setup"
```

Create the matching empty repository on GitHub. In the browser, click New repository, name it `python-for-network-engineers`, leave it empty (do not add a README from the website, since you already have local files), and create it. GitHub then shows you the repository URL. Connect your local repository to it and push:

```
git remote add origin https://github.com/YOUR-USERNAME/python-for-network-engineers.git
git branch -M main
git push -u origin main
```

The first push asks you to authenticate. In the browser sign-in flow, or when asked for a password in the terminal, use a personal access token, not your account password. You create a token in GitHub under Settings, Developer settings, Personal access tokens. Give it the `repo` scope, copy it, and paste it when Git asks for a password. Git remembers it after the first time.

Refresh your GitHub repository page. Your files are there. That is the full loop: write locally, commit, push to GitHub.

Code walkthrough

A few of these commands are worth understanding rather than just typing.

`python -m venv .venv` creates a folder named `.venv` that contains a private Python and a private place for packages. Nothing outside this folder is touched, which is exactly why isolation is safe.

Activating the environment does not change your files. It only changes which Python your shell uses, for this window, until you deactivate or close it. That is why the prompt marker matters, it is the only visible sign of which Python is active.

`git add .` moves your current changes into the staging area, and `git commit` records the staged changes as a permanent point in history with a message. The two steps are separate on purpose, so you can choose exactly what goes into each save. `git push` copies your local history up to GitHub. Look back at Figure 0.2 and match each command to an arrow.

Lab

Lab 0.1: From Zero to First Commit is provided as a worksheet at `labs/chapter00/lab00_zero_to_first_commit.md`.

Objective. Stand up a complete working environment and prove it end to end, from a fresh install to code pushed on GitHub.

Network scenario. You are joining a team that keeps all of its automation in GitHub. Before you can contribute a single script, your machine has to be set up and your first commit has to land in a repository.

Topology. None. This is all on your computer plus your GitHub account.

Prerequisites. A computer running Windows, macOS, or Linux, with administrator rights to install software, and internet access.

Files required. None to start. You create `requirements.txt`, `.gitignore`, and run `check_setup.py`.

Steps.

1. Install Python 3.14 for your operating system and confirm `python --version`.
2. Upgrade pip with `python -m pip install --upgrade pip`.
3. Create the `python-for-network-engineers` folder and a `.venv` virtual environment, then activate it.
4. Install IPython and run the interactive session shown in Part D.
5. Install VS Code and the Python extension.
6. Create a GitHub account and enable two-factor authentication.
7. Install Git and set your `user.name` and `user.email`.
8. Add `requirements.txt` and `.gitignore`, then run `git init`, `git add .`, and `git commit`.
9. Create the empty GitHub repository, add it as `origin`, and `git push`.
10. Copy `check_setup.py` into your `scripts/chapter00/` folder and run it.

Expected output. Running the setup check prints something like this (the path and release differ per machine):

```
Python version : 3.14.0
Operating system: Windows 11
Interpreter path: C:\Users\you\python-for-network-engineers\.venv\Scripts\python.exe
Result: your Python is new enough for this course.
```

And your GitHub repository page shows `requirements.txt`, `.gitignore`, and your first commit message.

Verification. You are done when three things are true: your prompt shows `(.venv)` when the environment is active, `check_setup.py` reports your Python is new enough, and your files appear on GitHub.

Common errors.

- `python is not recognized` on Windows. You skipped the "Add python.exe to PATH" checkbox. Re-run the installer, choose Modify, and enable it, or reinstall.
- Activation blocked on Windows. Run `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`, then activate again.
- `python: command not found` on macOS or Linux. Use `python3` and `pip3` until the environment is active.
- Push rejected or asks for a password that never works. You are using your account password. Create a personal access token with the `repo` scope and use that instead.
- You committed the `.venv` folder by mistake. Add `.venv/` to `.gitignore`, then run `git rm -r --cached .venv` and commit again.

Student exercise. See `labs/chapter00/exercise_second_commit.md`. You make a small change and record a second commit, to prove you can repeat the save loop on your own. The solution is in `solutions/chapter00/exercise_second_commit_solution.md`.

Challenge lab. See `labs/chapter00/challenge_branch_and_readme.md`. You create a branch, add a README on it, and push the branch to GitHub. The solution is in `solutions/chapter00/challenge_branch_and_readme_solution.md`.

> **Troubleshooting tip:** If `git push` hangs or fails, first run `git remote -v` and confirm the `origin` URL is correct and points at your own username. A typo in the remote URL is the usual cause.

Key takeaways

Install the latest Python and always work inside a per-project virtual environment, so installs never collide and your prompt shows `(.venv)` when you are set. pip installs the libraries the course needs, tracked in `requirements.txt`. VS Code is where you write code and IPython is where you try it line by line. Git records the history of your files locally, and GitHub keeps a copy in the cloud that you can restore and share. The save loop is add, commit, push, and you will run it for the rest of the course. GitHub authentication uses a token or an SSH key, never your password.

Review questions

1. What problem does a virtual environment solve, and how can you tell one is active?
2. What is the difference between Git and GitHub?
3. Why do you run `python -m pip` instead of just `pip`?
4. What is the purpose of `.gitignore`, and name two things that belong in it.
5. On Windows, which single installer choice prevents the "python is not recognized" error?
6. Put these in order and say what each does: `git push`, `git commit`, `git add`.
7. Why should the virtual environment folder never be committed to Git?
8. When Git asks for a password to push to GitHub, what should you actually provide?

Interview questions

1. Walk me through how you set up a clean Python environment for a new automation project.
2. Why is version control important for network automation specifically, not just software development?
3. A teammate says their script works on their laptop but fails on yours. What environment issues would you check first?
4. What is a virtual environment, and what goes wrong without one?
5. How do you keep secrets like device passwords out of a Git repository?

> **Interview tip:** Mentioning virtual environments, a pinned `requirements.txt`, and keeping secrets out of Git signals that you think about repeatability and safety, which is what separates a hobby script from a tool a team can rely on.

Repository files for this chapter

- `docs/chapters/chapter00_prerequisites.md` (this chapter)
- `docs/diagrams/ch00_setup_pipeline.svg` (Figure 0.1)
- `docs/diagrams/ch00_git_github_flow.svg` (Figure 0.2)
- `scripts/chapter00/check_setup.py` (environment check)
- `labs/chapter00/lab00_zero_to_first_commit.md` (lab worksheet)
- `labs/chapter00/exercise_second_commit.md` (student exercise)
- `labs/chapter00/challenge_branch_and_readme.md` (challenge lab)
- `solutions/chapter00/exercise_second_commit_solution.md`
- `solutions/chapter00/challenge_branch_and_readme_solution.md`

What is next

Chapter 1 explains why Python is worth learning for network work and shows the payoff before you write any code. Because your environment is now ready, you can run the demo in Chapter 1 straight away. Chapter 2 then starts the Python language itself with your first programs.

Chapter 1: Why Python for Network Engineers

> **Part 1: Python Foundations for Network Engineers** This chapter is about the why, not the how. It assumes you know nothing about Python, and it does not ask you to read or write any real code. You start the language itself in Chapter 2, from a single line, and build up one small idea at a time.

Learning objectives

After this chapter you will be able to:

- Explain in plain words why manual command line work stops scaling as a network grows.
- Name the everyday network tasks that Python takes the pain out of.
- Compare Python with Bash, Ansible, and device APIs, and say when each one is the right tool.
- Place Python correctly in a modern network automation workflow.
- Recognize what a single line of Python looks like, without needing to write any yet.

Why this matters to a network engineer

Picture a normal Tuesday. You have a change window at six in the morning, fifty branch routers to touch, and a rollback plan that only works if every device was configured exactly the same way. You SSH into the first router, paste the config, check the output, move to the next one. Somewhere around router thirty your coffee is cold, your eyes are tired, and you fat-finger a VLAN number. Now one site is different from the other forty-nine, and nobody will notice until it breaks two weeks later during an audit.

None of that is a skills problem. It is a scale problem. A human doing the same task by hand, over and over, will eventually make a small mistake, and small mistakes in a network turn into outages. The work is also slow, it depends on one person being awake, and when someone asks "what was the state of every BGP session last Friday at 9am," you have no record to show them.

Python does not make you a better engineer. You already know the network. What it does is take the parts of the job that are repetitive, error prone, and slow, and do them the same way every time, across as many devices as you want, in seconds, with a log of exactly what happened. That is the whole pitch. Everything in this course builds toward it.

Concept

Automation here means something very simple. Anything you can do by typing at a device prompt, Python can do for you. It opens a session to the device, sends the same commands you would type, reads the text that comes back, and then does something useful with that text. The difference is that it never gets tired, never skips a device, and never types `vlan 201` when it meant `vlan 210`.

Traditional command line operation looks like this: connect, type a command, read the result with your eyes, decide what it means, and repeat for the next device. That loop is fine for one or two devices. It becomes painful at fifty and impossible to do consistently at five hundred. The reasons are always the same.

Manual work is repetitive, so it wastes your time on tasks a machine should own. It is inconsistent, because two engineers, or the same engineer on two different days, will not do the check in exactly the same way. It is error prone, because typing the same thing a hundred times invites a typo. It leaves no record, so you cannot prove what the network looked like at a point in time. And it does not scale, because the effort grows in a straight line with the number of devices.

> **Important note:** Python does not replace your networking knowledge. It gives that knowledge reach. You still decide what "healthy" means for a BGP session or an interface. Python just applies your decision everywhere, at once.

Network example

Here are six tasks that show up in almost every network job. You will build a real, working version of each one during this course. Read them now as a preview of what Python is good for.

Checking one hundred routers. Instead of opening one hundred SSH sessions, one script connects to all of them and reports which are reachable and which are not.

Backing up configurations. Rather than copying and pasting running configs into files by hand, a script pulls the config from every device and saves each one with a timestamp, every night, without you being there.

Finding interface errors. A script collects interface counters from the whole fleet and shows only the interfaces with rising error rates, so you look at the five that matter instead of scrolling through thousands of lines.

Checking BGP neighbors. A script asks every router about its BGP sessions and flags any neighbor that is not in the established state, so you find the broken peering before your customer does.

Bulk VLAN creation. Instead of adding the same VLAN on forty switches by hand, a script pushes it to all of them from one list, the same way each time.

Validating IP addresses. Before you deploy anything, a script checks that every address in your plan is a real, correctly formatted address in the right subnet, so a bad entry is caught on your laptop and not on the router.

> **Example:** Later in this chapter you will read a small script that does a simplified version of the interface check. It takes a table of interface states and pulls out only the ones that are supposed to be up but are actually down.

How it works

Under the hood, every one of those tasks is built from two halves.

The first half is the connection. Python uses a library to open a session to the device. Depending on the device and what it supports, that session might be SSH to a command line, or a REST API call over HTTPS, or a NETCONF session, or a gNMI stream. You do not have to build any of that yourself. You pick a library, give it an address and credentials, and it hands you a connection.

The second half is the logic. Once the command output comes back as text, you decide what to do with it. You might search it for a pattern, count something, compare it to what you expected, save it to a file, or raise an alert. This is where your networking judgment lives, expressed as a few lines of Python.

That split is worth remembering, because the whole course follows it. The device connection chapters (netmiko, scrapli, REST, NETCONF) teach the first half. The data handling chapters (data types, files, regular expressions, parsing) teach the second half. Put the two together and you have automation.

Visual explanation

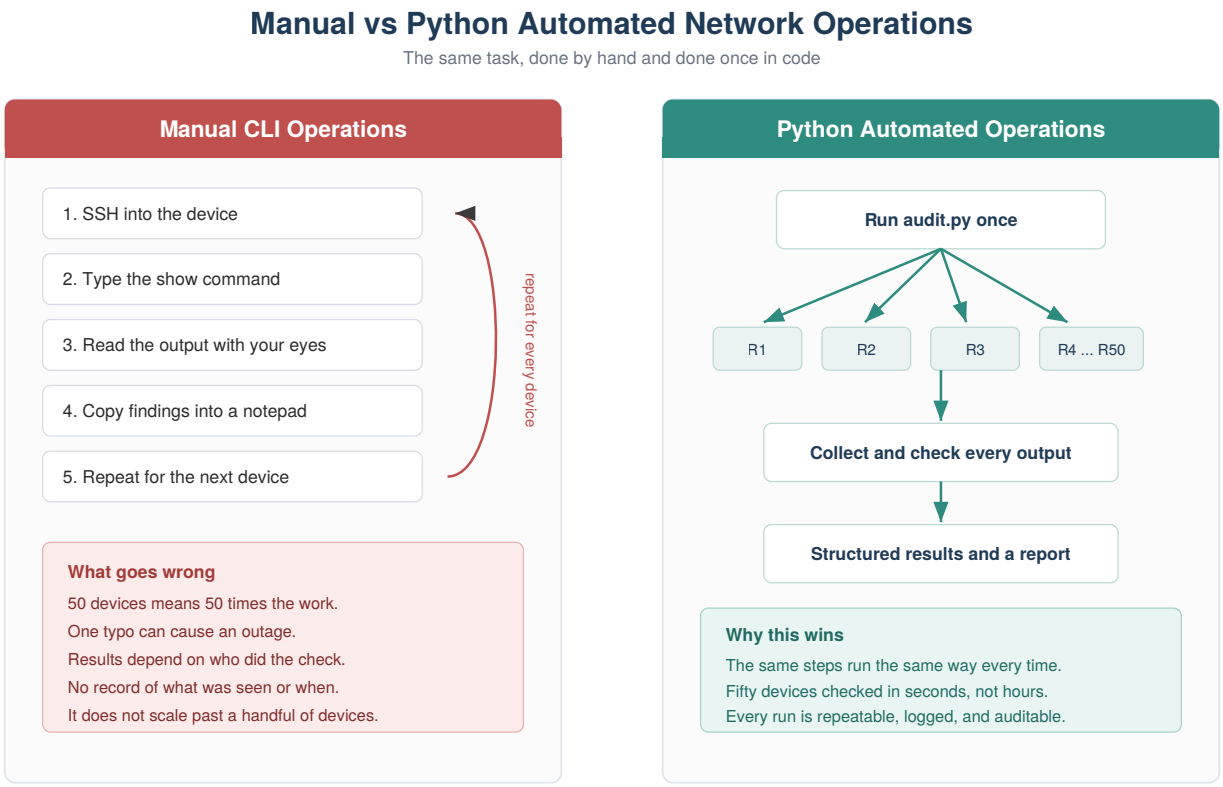

Figure 1.1: Manual vs Python automated network operations. On the left, the same steps are repeated by hand for every device, and the effort and risk grow with the device count. On the right, one script fans out to the whole fleet and returns a structured, repeatable result.

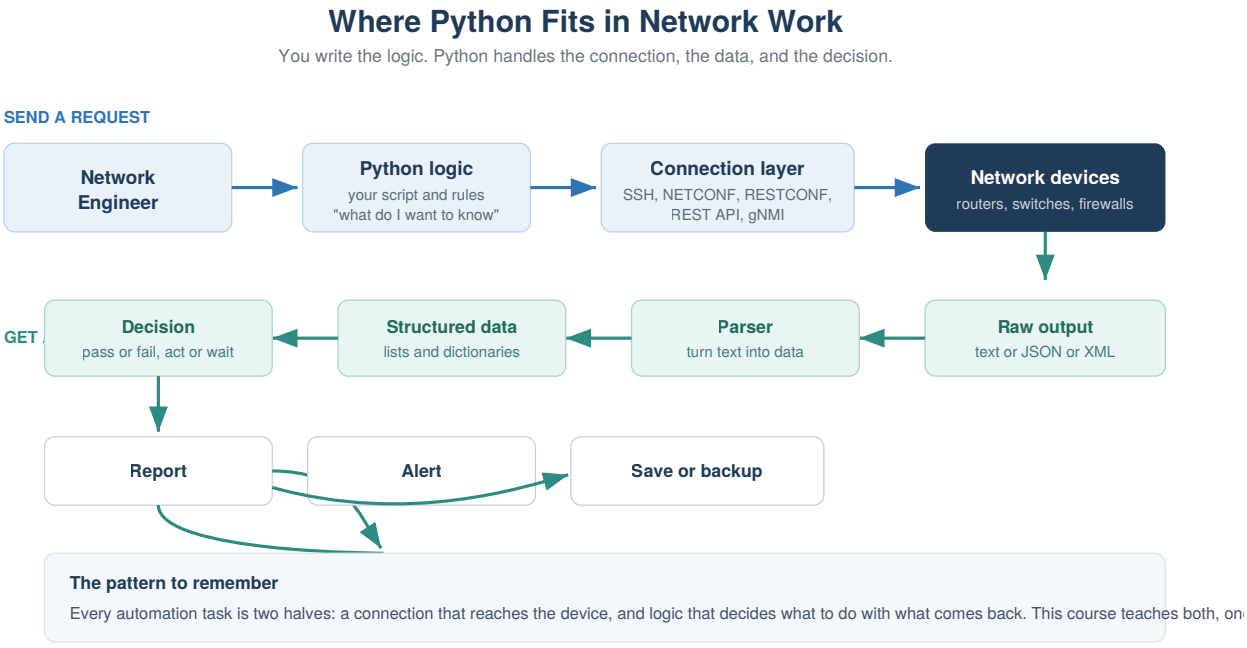

Figure 1.2: Where Python fits. You supply the logic. Python handles the connection to the device, turns the raw output into data you can work with, makes the decision you defined, and produces a report, an alert, or a saved backup.

What Python code looks like

You do not know any Python yet, and that is fine. This book does not ask you to read a real program in this chapter. You start writing code in Chapter 2, from the very first line, and every new idea after that uses only what you have already learned.

So the word code is not a mystery, here is the smallest possible Python program:

```
print("Starting the interface check")
```

That is one complete line. The word `print` is a built-in instruction that means "show this on the screen." The text inside the quotes is what gets shown. When you run it, you see exactly this:

```
Starting the interface check
```

That single idea, telling Python to show something, is where Chapter 2 begins. From there you learn to store a value, then to show several values together, then to make a simple decision, and only later to repeat an action across many devices. Each step is small and rests on the one before it. By the time you reach a program that checks the interfaces on fifty routers, you will have written every part of it yourself and understood each line as you added it.

> **Important note:** There is an optional look-ahead script in the repository, `scripts/chapter01/interface_audit_demo.py`, that shows these ideas working together in a fuller program. You are not expected to understand it yet, and you can safely ignore it for now. Come back to it after Chapter 5 and it will read like plain English.

Python compared with the other tools

Python is not the only way to automate a network, and it is not always the best one. Knowing when to reach for something else is part of using it well.

Bash is excellent for short, local, glue tasks: renaming files, chaining a few commands, scheduling a job. It gets awkward the moment you need to reach into command output, make decisions on structured data, or handle errors cleanly. Python handles all of that comfortably, so the rule of thumb is Bash for a few lines of plumbing, Python once there is real logic.

Ansible is a configuration management tool with many ready-made network modules. It is great for pushing a known desired state to devices with little to no code, and teams like it because playbooks read almost like a checklist. It is less suited to tasks with lots of custom

logic, complex parsing, or anything that does not fit its model. A common and healthy setup is Ansible for straightforward config pushes and Python for everything with real logic behind it. They are not rivals.

Device APIs (REST, NETCONF, RESTCONF, gNMI) are not an alternative to Python, they are a better door into the device. Older gear only offers a command line, so you automate by sending CLI commands and reading text back. Newer gear offers APIs that return structured data directly, which is cleaner and more reliable. Python is how you talk to those APIs. So the comparison is really Python plus CLI versus Python plus an API, and you will learn both, because real networks are a mix of old and new.

Table 1.1: When to use each tool

Tool	Best at	Weak at	Typical use with a network
Bash	Short local glue, scheduling	Logic, parsing, error handling	Wrapping and scheduling scripts
Ansible	Pushing known config state	Heavy custom logic and parsing	Standard config rollout
Device APIs	Returning structured data	Nothing, but not on old gear	The cleanest way in, driven by Python
Python	Logic, data, glue, scale	Being the shortest possible one-liner	The general purpose automation language

> **Best practice:** Do not think of these as competitors. Mature automation setups use Bash to schedule, Python for logic, Ansible for standard pushes, and APIs wherever the hardware supports them. Python is the piece that ties the rest together.

Where Python fits in network automation

If you zoom out, a modern automation setup has a few layers, and Python threads through most of them. There is a source of truth that holds what the network should look like (inventory, addresses, intended config). There is a connection layer that reaches the devices. There is logic that collects state, compares it to intent, and decides what to do. And there is output: reports, alerts, backups, and changes.

You will build a small version of every one of those layers in this course, and by the capstone you will connect them into a single tool. For now, just hold the shape in your head. Python is the glue and the brains. The devices, the APIs, and the data formats are the parts it connects.

Lab

Lab 1.1: Count the Cost of Manual Work is a thinking lab, provided as a worksheet at `labs/chapter01/lab01_cost_of_manual_work.md`. It needs no computer and no code. The point is to feel, in numbers, why automation is worth learning.

Objective. Estimate the time and error cost of a manual daily check, and compare it to an automated one.

Network scenario. Every morning your team checks that the WAN interface is up on each branch router.

Prerequisites. None. Bring a pen.

Steps.

1. Suppose you have 60 branch routers, and it takes about 45 seconds to log in, run one command, read the result, and log out of each one. How long does the full manual check take each morning?
2. Over a five-day week, how much time is that?
3. People make small mistakes when doing the same thing over and over. If you misread just one router in fifty, roughly how many misreads might you make across a month of daily checks?
4. Now assume an automated check runs in about 30 seconds for all 60 routers and never misreads. How much time does it save per week?
5. Write two or three sentences on what you would do with the time saved.

Expected result. Rough numbers are fine. The manual check is about 45 minutes a day, close to four hours a week. The automated check is under a minute. The saving is large, and the misread rate on the reading step drops to zero.

Verification. You have a per-week time saving and a clear sense of how errors grow as a manual task is repeated.

Student exercise. List five tasks you do by hand today that you would like to automate. For each one, write down the manual steps you follow. Keep this list. As the course goes on you will build tools for several of them. A sample answer is in `solutions/chapter01/exercise_manual_tasks.md`.

Challenge. From your list, pick the task that would save the most time if automated, and write down how you would know the automation worked correctly. Thinking about how to check the result, before you build anything, is a habit that separates safe automation from risky automation.

> **Best practice:** Keep the list from the exercise somewhere you will see it. It becomes your personal backlog, and finishing this course means turning several of those manual chores into tools you trust.

Key takeaways

Manual network operations do not scale, and the failures are consistency and human error, not lack of skill. Python takes the repetitive parts and does them the same way every time, across the whole fleet, with a record of what happened. Every automation task is two halves, a connection to the device and logic about the output, and the whole course is organized around teaching both. Python is not in competition with Bash, Ansible, or device APIs. It is the general purpose language that ties them together. You do not need to write code yet. You need to see the destination, which is a network you manage through tested, repeatable programs instead of by hand.

Review questions

1. Give two concrete reasons manual CLI operations become risky as a network grows.
2. What are the two halves that every automation task is built from?
3. What does the `print` instruction do?
4. In plain words, what is the payoff of automating a daily check across many devices?
5. When would you reach for Bash instead of Python, and when for Python instead of Bash?
6. Are device APIs a replacement for Python? Explain your answer.
7. Name three of the six preview tasks and, in one line each, say what Python does for them.
8. Why is it a good habit to decide how you will check an automation before you build it?

Interview questions

1. Your team still does a morning health check by hand across sixty devices. How would you make the case for automating it, and what would you automate first?
2. When would you choose Ansible over a custom Python script, and when the reverse?
3. A colleague says APIs make Python unnecessary for network automation. How do you respond?
4. What are the risks of automation done badly, and how would you reduce them?
5. Explain the difference between a device's raw CLI output and structured data, and why the difference matters for automation.

> **Interview tip:** Interviewers care less about syntax here and more about judgment. Show that you know automation is about consistency and safety, that you would start small and prove value, and that you pick the tool that fits the task rather than forcing one tool onto everything.

Repository files for this chapter

- `docs/chapters/chapter01_why_python.md` (this chapter)

- `docs/diagrams/ch01_manual_vs_automated.svg` (Figure 1.1)
- `docs/diagrams/ch01_where_python_fits.svg` (Figure 1.2)
- `labs/chapter01/lab01_cost_of_manual_work.md` (lab worksheet)
- `solutions/chapter01/exercise_manual_tasks.md` (sample exercise answer)
- `scripts/chapter01/interface_audit_demo.py` (optional look-ahead, understandable after Chapter 5)
- `labs/chapter01/exercise_count_admin_down.py` (student exercise)
- `labs/chapter01/challenge_group_by_device.py` (challenge lab)
- `solutions/chapter01/lab01_answers.md`
- `solutions/chapter01/exercise_count_admin_down_solution.py`
- `solutions/chapter01/challenge_group_by_device_solution.py`

What is next

Chapter 2 begins the Python language itself. You write your first real programs, storing device facts in variables, printing clean output, and converting between text and numbers, all with networking examples. Your environment is already set up from Chapter 0, so you can run everything as you go.

Chapter 2: Your First Python Programs

> **Part 1: Python Foundations for Network Engineers** This is where you actually start writing Python. It assumes you have never programmed before. We begin with one line, `print`, and add one small idea at a time: comments, variables, text, numbers, true or false values, and how to turn one into another. Every example uses network data you already understand.

Learning objectives

After this chapter you will be able to:

- Show text on the screen with `print`.
- Write comments to leave notes in your code.
- Store a value in a variable and use it later.
- Tell apart the four everyday value types: strings, integers, floats, and booleans.
- Convert between text and numbers, and explain why that is necessary.
- Ask the user for input.
- Combine variables and text neatly with f-strings.

Why this matters to a network engineer

Everything you automate later is built from these small pieces. When a script connects to a router, the hostname it uses is stored in a variable. When it reads a port number from a file, that value arrives as text and has to be turned into a number before you can do math with it. When it prints a report, it uses f-strings to drop values into neat lines. None of this is advanced, but all of it shows up in every real tool. Get comfortable here and the rest of the course is much easier.

How to work through this chapter

Keep two things open: an IPython shell to try single lines, and VS Code to write scripts you save and run. When you see a block like the one below, it is an IPython session. `In [1]:` is what you type. `Out[1]:` is what Python shows back.

```
In [1]: 2 + 2
Out[1]: 4
```

Type the examples yourself. Reading them is not the same as running them.

Showing text with print

The simplest useful thing a program can do is show something on the screen. In Python that is the `print` instruction. Put what you want to show inside the parentheses.

```
In [1]: print("Hello from your first Python program")
Hello from your first Python program
```

Notice there is no `Out[]` line here. `print` shows text as a side effect, it does not hand a value back. You can print more than one line by calling `print` more than once. Save this as a file called `hello.py` and run it:

```
print("Hello from your first Python program")
print("You are about to automate your network")
```

Run it from your terminal (with your virtual environment active) using `python hello.py`. You get:

```
Hello from your first Python program
You are about to automate your network
```

That is a complete program. Two instructions, run top to bottom.

Comments

A comment is a note for humans that Python ignores. Anything after a `#` on a line is a comment. Use comments to explain why something is there, not to repeat what the code obviously does.

```
# Print a banner before the audit starts
print("Starting the interface audit")
```

> **Best practice:** Good comments explain intent. A comment like `# add 1 to x` is noise. A comment like `# skip the management interface, we never touch it` earns its place.

Variables

Typing the same value over and over is a waste, and it makes changes painful. A variable lets you store a value under a name you choose, then use that name wherever you need the value. You create one with a single equals sign: the name on the left, the value on the right.

```
In [1]: hostname = "core-sw-01"

In [2]: print(hostname)
core-sw-01
```

After that first line, writing `hostname` anywhere is the same as writing `"core-sw-01"`. If the device is renamed, you change one line instead of hunting through the whole program.

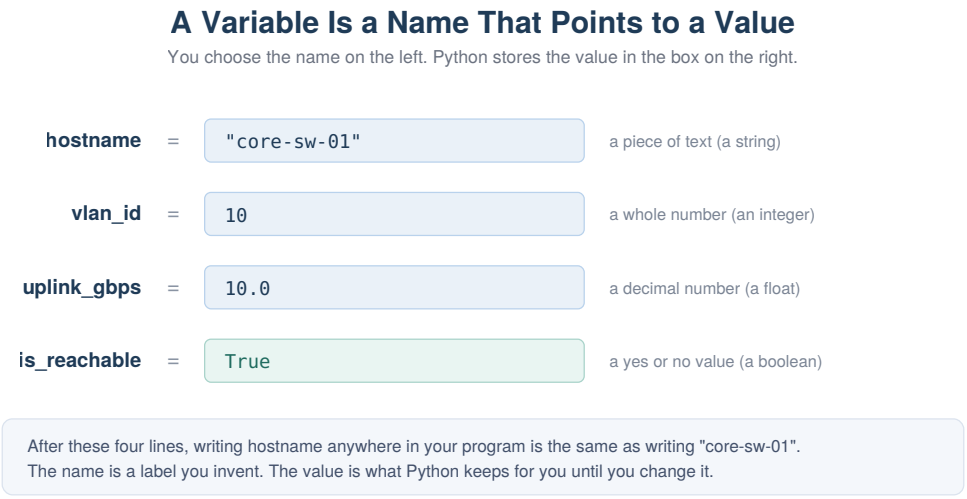

Figure 2.1: A variable is a name you invent on the left and a value Python stores on the right. Later, using the name gives you back the value.

A few rules for names. Use lowercase letters and underscores, like `vlan_id` or `mgmt_ip`. Names can contain letters, numbers, and underscores, but cannot start with a number and cannot contain spaces. Pick names that say what the value is. `router_ip` is better than `x`.

> **Best practice:** A good variable name is a small gift to the next person who reads your code, which is often you, three months later. `bgp_neighbor` beats `bn`.

The four everyday value types

Early on, almost every value you store is one of four types. You do not declare the type, Python works it out from how you write the value.

The Four Everyday Value Types

Almost everything you store early on is one of these four

Type	What it holds	How to spot it	Network example
string (str)	text of any kind	`wrapped in quotes`	`"GigabitEthernet0/1"` `"10.0.0.1"`
integer (int)	a whole number	`no quotes, no dot`	`65001 (an ASN)`
float	a decimal number	`has a dot`	`10.0 (Gbps)`
boolean (bool)	a yes or no	`True or False`	`True (is it up?)`

The quotes are the key tell. "10" is text. 10 is a number you can do math with.

Figure 2.2: Strings, integers, floats, and booleans, with how to recognize each and a network example of each.

A string is text. You write it inside quotes, single or double, it does not matter as long as they match. Device names, interface names, and yes, IP addresses, are all text.

```
In [1]: hostname = "core-sw-01"

In [2]: interface = "GigabitEthernet0/1"
```

An integer is a whole number, written with no quotes and no decimal point. VLAN IDs and AS numbers are integers.

```
In [3]: vlan_id = 10

In [4]: asn = 65001
```

A float is a number with a decimal point. Link speeds or utilization percentages are often floats.

```
In [5]: uplink_gbps = 10.0
```

A boolean is a yes or no value, written as `True` or `False` with a capital first letter. It answers a question: is this interface up, is this device reachable.

```
In [6]: is_reachable = True
```

You can ask Python what type a value is with `type`. This is handy when you are not sure.

```
In [7]: type("core-sw-01")
Out[7]: <class 'str'>

In [8]: type(10)
Out[8]: <class 'int'>

In [9]: type(10.0)
```

```
Out[9]: <class 'float'>

In [10]: type(True)
Out[10]: <class 'bool'>
```

The quotes are the key tell. `"10"` is text. `10` is a number. They look almost the same to you, but Python treats them very differently, as you are about to see.

Doing a little math

Numbers behave the way you expect. The usual symbols are `+`, `-`, `*` for multiply, and `/` for divide.

```
In [1]: access_ports = 24

In [2]: uplink_ports = 2

In [3]: access_ports + uplink_ports
Out[3]: 26
```

Text does not add like numbers. Putting a `+` between two strings joins them end to end, which is occasionally useful but often a trap.

```
In [4]: "Gig" + "0/1"
Out[4]: 'Gig0/1'

In [5]: "24" + "2"
Out[5]: '242'
```

Look at `In [5]` carefully. `"24"` and `"2"` are text, so `+` glued them into `"242"`, not `26`. This is the single most common beginner surprise, and it matters a lot in network work, because values that come from files, forms, and device output almost always arrive as text.

Converting between types

To move between text and numbers you use conversion functions: `int()` makes a whole number, `float()` makes a decimal, and `str()` makes text.

```
In [1]: int("22")
Out[1]: 22

In [2]: int("22") + 1
Out[2]: 23

In [3]: str(22)
Out[3]: '22'

In [4]: "port " + str(22)
Out[4]: 'port 22'
```

So the rule is simple. If a number arrived as text and you need to do math, convert it with `int()` or `float()` first. If you have a number and you want to build a message, convert it to text with `str()`, or better, use an f-string, which does the conversion for you.

> **Warning:** `int("Gig0/1")` fails, because that text is not a number. Only convert text that really is a number. You will learn to handle failures safely in the error handling chapter.

Getting input from the user

Sometimes you want to ask the person running the script for a value. The `input` instruction shows a prompt and hands back whatever they type, always as text.

```
hostname = input("Enter the device hostname: ")
print("You entered: " + hostname)
```

Because `input` always returns text, a number typed by the user is text until you convert it:

```
vlan_text = input("Enter the VLAN ID: ")
vlan_id = int(vlan_text)          # turn the text into a real number
print(f"VLAN {vlan_id} will be created")
```

f-strings: putting values into text neatly

Joining text and values with `+` and `str()` gets clumsy fast. An f-string is the clean way. Put the letter `f` right before the opening quote, then write your text and drop any variable inside curly braces.

```
In [1]: hostname = "core-sw-01"

In [2]: vlan_count = 12

In [3]: f"{hostname} carries {vlan_count} VLANs"
Out[3]: 'core-sw-01 carries 12 VLANs'
```

Python replaces `{hostname}` with the value of `hostname` and `{vlan_count}` with its value, converting numbers to text automatically. f-strings are how you will build almost every message and report in this course.

Code walkthrough: a device summary

Here is a small program that pulls the chapter together. It stores the facts about one device and prints a tidy summary. It uses only what you have learned: variables, the four value types, and f-strings. This is `scripts/chapter02/device_banner.py`.

```
hostname = "core-sw-01"
mgmt_ip = "10.0.0.1"
vlan_count = 12
uplink_gbps = 10.0
is_reachable = True

print("Device summary")
print("==============")
print(f"Hostname     : {hostname}")
```

```
print(f"Management IP: {mgmt_ip}")
print(f"VLAN count   : {vlan_count}")
print(f"Uplink speed : {uplink_gbps} Gbps")
print(f"Reachable    : {is_reachable}")
```

Why is it written this way? Each fact gets its own well named variable, so the program reads like a description of the device. The address is stored as text, because an IP address is not something you do math on, it is a label. The VLAN count is an integer and the uplink speed is a float, because those are numbers. The reachable flag is a boolean, because it answers a yes or no question. The printing uses f-strings so each value drops neatly into a labeled line. If you had ten devices you would not copy this ten times, you would use a loop, which is Chapter 4. One device at a time is the right size for now.

Run it and you get:

```
Device summary
==============
Hostname     : core-sw-01
Management IP: 10.0.0.1
VLAN count   : 12
Uplink speed : 10.0 Gbps
Reachable    : True
```

Lab

Lab 2.1: Build a Device Banner is provided as a worksheet at `labs/chapter02/lab02_device_banner.md`.

Objective. Write your first useful script from scratch, using variables, value types, and f-strings to print a clean summary for one device.

Network scenario. You want a quick, repeatable way to print a labeled summary of a device's key facts, the kind of thing you would later generate for every device in your network.

Topology. None. Everything runs on your computer.

Prerequisites. Chapter 0 finished, so Python and your virtual environment are ready.

Files required. None to start. You create `device_banner.py`.

Steps.

1. In your project, create a file `scripts/chapter02/device_banner.py`.
2. Create five variables for a device: `hostname` and `mgmt_ip` as text, `vlan_count` as an integer, `uplink_gbps` as a float, and `is_reachable` as a boolean.
3. Print a two line header.
4. Print each fact on its own line using an f-string, with aligned labels.
5. Save and run it with `python scripts/chapter02/device_banner.py`.

Complete script. See the walkthrough above, which is the full solution.

Expected output.

```
Device summary
==============
Hostname     : core-sw-01
Management IP: 10.0.0.1
VLAN count   : 12
Uplink speed : 10.0 Gbps
Reachable    : True
```

Verification. Your output matches, and each value sits after its label. If a number came out with quotes around it, you stored it as text by mistake.

Common errors.

- `NameError: name 'hostname' is not defined`. You used a variable before creating it, or misspelled the name. Names must match exactly.
- The IP address shows as a number error. Keep the address in quotes. It is text.
- You forgot the `f` before the quotes, so the line printed `{hostname}` literally instead of the value. Add the `f`.

Student exercise. Copy your script to make a banner for a second, different device (a different hostname, address, VLAN count, speed, and a `False` reachable flag). The solution is in `solutions/chapter02/device_banner_edge_solution.py`.

Challenge lab. Imagine the access and uplink port counts arrive as text, for example `"24"` and `"2"`. Write a short script that converts both to integers, adds them, and prints the total number of ports. Then, in a comment, explain what `"24" + "2"` would print instead and why. The solution is in `solutions/chapter02/challenge_port_count_solution.py`.

> **Troubleshooting tip:** If your total came out as `242`, you added the values as text. Convert each with `int()` before adding.

Key takeaways

`print` shows text on the screen. A `#` starts a comment, a note Python ignores. A variable stores a value under a name you choose, using a single `=`. The four everyday types are strings (text in quotes), integers (whole numbers), floats (decimals), and booleans (`True` or `False`). Text and numbers are not the same, so `"24" + "2"` is `"242"`, not `26`. Use `int()`, `float()`, and `str()` to convert, and remember that `input` always gives you text. f-strings are the clean way to put values into text. Everything in this chapter is small, and everything later is built from it.

Review questions

1. What is the difference between `print` and a comment?
2. Write the line that stores the text `Loopback0` in a variable called `interface`.
3. What are the four everyday value types, and how do you recognize each one?

4. What does `"22" + "1"` produce, and what does `int("22") + 1` produce? Why are they different?
5. Why does `input` always return text, and what do you do about it when you need a number?
6. Rewrite `"Device " + hostname + " has " + str(vlan_count) + " vlans"` as an f-string.
7. What error do you get if you use a variable you never created, and what usually causes it?
8. Is `"10.0.0.1"` a string or a number in Python, and why does that make sense?

Interview questions

1. A value read from a CSV file is `"65001"` and you need to use it as an AS number in a comparison. What do you do first?
2. Explain the difference between an integer, a float, and a string, with a network example of each.
3. Why might storing an IP address as a string be the right choice rather than a number?
4. What is an f-string, and why is it preferred over joining text with plus signs?

> **Interview tip:** The text versus number distinction sounds trivial, but it is behind a large share of real automation bugs. Showing that you always think about what type a value is, especially data that came from a file or a device, signals care.

Repository files for this chapter

- `docs/chapters/chapter02_first_programs.md` (this chapter)
- `docs/diagrams/ch02_variables_boxes.svg` (Figure 2.1)
- `docs/diagrams/ch02_value_types.svg` (Figure 2.2)
- `scripts/chapter02/hello.py` (first program)
- `scripts/chapter02/device_banner.py` (device summary)
- `labs/chapter02/lab02_device_banner.md` (lab worksheet)
- `solutions/chapter02/device_banner_edge_solution.py` (exercise solution)
- `solutions/chapter02/challenge_port_count_solution.py` (challenge solution)

What is next

Chapter 3 answers the question you may already be asking: what if I have not one device but fifty? That is where data structures come in. You will learn what a data structure is in plain terms, then meet the list and the dictionary, which let you hold many values and many labeled facts together. Those two are the backbone of every real network script.